EA2. Creación de Repositorio:
Procesamiento Funcional de Datos para el Sistema de Transporte TecnoMovil Data

David Esteban Zapata Rodríguez

Grupo 10

Institución Universitaria Digital de Antioquia

Ingeniería de Software y Datos

Programación Orientada a Objetos II

Docente: Jorge Armando Julio

2026

Tabla de Contenido

1. Introducción

2. Objetivos

3. Desarrollo

    1. Análisis del Caso de Estudio

    2. Diseño de la Solución

    3. Generación de Datos Simulados

    4. Implementación de las Tareas a) a f)

    5. Conceptos de Programación Funcional Aplicados

    6. Pruebas del Código

    7. Evidencia de Ejecución

    8. Repositorio en GitHub y Archivo ZIP

4. Conclusiones

5. Enlaces del Repositorio

6. Referencias

7. Anexo. Código Fuente Completo

Introducción

La ciudad de TecnoValle mueve cada día a más de 450.000 pasajeros en buses, rutas alimentadoras y estaciones. Su sistema de transporte genera entre 6 y 12 millones de registros diarios, pero el programa que los procesa fue hecho con un enfoque imperativo: recorre los datos uno por uno con ciclos, cambia variables compartidas y tarda hasta dos horas en procesar un día. Además, como varias partes del programa modifican las mismas listas al mismo tiempo, a veces los conteos salen mal.

Para resolverlo, la alcaldía creó el proyecto TecnoMovil Data, que busca rehacer el módulo de procesamiento con *programación funcional*. En este estilo, en lugar de decirle al computador paso a paso **cómo** hacer algo (ciclos, contadores, variables que cambian), se le dice **qué** se quiere obtener, usando funciones que no cambian los datos que reciben. En Java esto se logra con lambdas, Streams, funciones de orden superior y colecciones inmutables (Urma et al., 2018).

En este informe se presenta el módulo desarrollado en Java, sin frameworks externos, que resuelve las seis tareas del caso de estudio: afluencia por estación, horas pico, rutas más usadas, patrones de viaje, tiempos entre estaciones y detección de rutas sobrecargadas. Se explica cómo se aplicó cada concepto funcional, se muestran las pruebas y la ejecución con más de 500.000 registros simulados, y se entrega el enlace del repositorio en GitHub donde está el proyecto.

Objetivos

Objetivo General

Desarrollar en Java un módulo de procesamiento de datos de transporte urbano usando programación funcional (funciones puras, inmutabilidad, lambdas, funciones de orden superior y Streams), y publicarlo en un repositorio de GitHub.

Objetivos Específicos

- Aplicar los conceptos de programación funcional a un caso realista: el sistema de transporte de TecnoValle.
- Procesar un volumen grande de datos simulados (más de 500.000 registros) de forma declarativa con Streams.

- Implementar las seis tareas del caso de estudio con funciones puras, lambdas y operaciones de orden superior, sin modificar la lista original.
- Comprobar que el procesamiento en paralelo con `parallelStream()` da exactamente los mismos resultados que el secuencial.
- Verificar el funcionamiento del código con pruebas propias, sin frameworks externos.
- Crear un repositorio en GitHub y alojar la solución en la rama principal (*main*).

Desarrollo

Análisis del Caso de Estudio

Cada registro del sistema tiene cinco datos: el usuario, la ruta, la estación, la acción (*entrada* o *salida*) y la fecha y hora (*timestamp*). La Tabla 1 muestra cómo se interpretó cada tarea pedida y qué método la resuelve.

Tabla 1

Tareas del caso de estudio y forma de resolverlas

Tarea	Cómo se interpretó	Método
a) Afluencia por estación	Contar las entradas de cada estación en el día. También se cuentan los usuarios distintos (un usuario que entra dos veces cuenta una sola vez).	`afluenciaPorEstacion` `usuariosUnicosPorEstacion`
b) Horas pico	Agrupar todos los registros (entradas y salidas) por la hora del día y escoger las horas con más movimiento.	`flujoPorHora` `horasPico`
c) Rutas más utilizadas	Contar las entradas de cada ruta y ordenarlas de mayor a menor.	`usosPorRuta` + `topN`
d) Patrones de viaje	Para cada usuario, la lista de estaciones en el mismo orden en que aparecen en los registros.	`patronesPorUsuario`
e) Tiempo entre estaciones	Ordenar los registros de cada usuario por hora, tomar los pares consecutivos de entrada → salida en estaciones distintas y calcular los minutos entre ellos.	`tiemposEntreEstaciones`
f) Sobrecarga en rutas	Buscar la hora con más entradas de cada ruta, dividirla por una capacidad simulada (4.500 pasajeros/hora) y marcar la ruta como “crítica” si pasa del 85 %.	`estadoDeRutas`

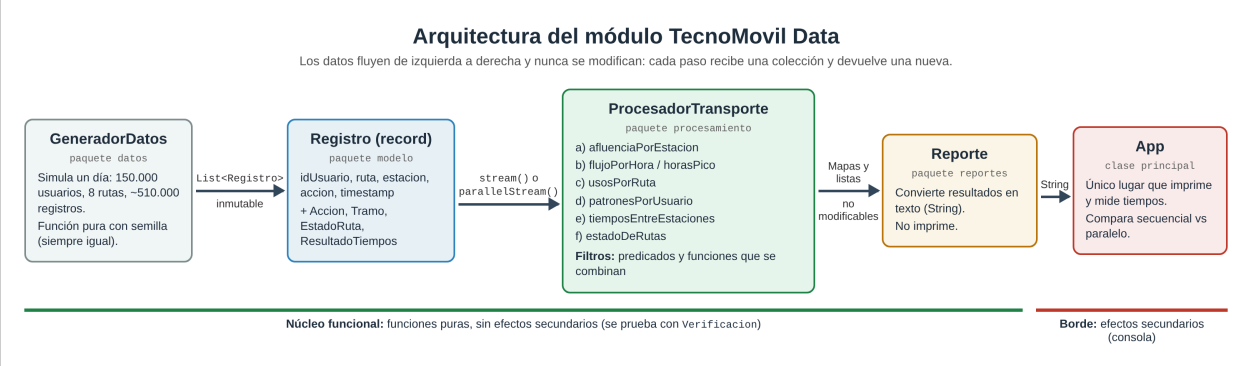
Nota. Todos los métodos están en la clase ProcesadorTransporte.

Diseño de la Solución

El proyecto se organizó en paquetes, cada uno con una sola responsabilidad (Figura 1). La idea principal es separar el **núcleo funcional**, donde se hacen los cálculos con funciones puras, del **borde**, que es la única parte que tiene efectos secundarios: imprimir en pantalla y medir el tiempo. Así, todos los cálculos se pueden probar y ejecutar en paralelo sin riesgo.

Figura 1

Arquitectura del módulo TecnoMovil Data



Nota. Elaboración propia.

Tabla 2

Paquetes y clases del proyecto

Paquete	Clase	Responsabilidad
`modelo`	Registro, Accion, Tramo, EstadoRuta, ResultadoTiempos	Datos inmutables (records y enum).
`datos`	GeneradorDatos	Crear los datos simulados de un día.
`procesamiento`	Filtros	Predicados y funciones reutilizables que se combinan.
`procesamiento`	ProcesadorTransporte	Las seis tareas del caso de estudio.
`reportes`	Reporte	Convertir los resultados en texto.
(raíz)	App	Programa principal: ejecuta todo e imprime.
(raíz)	Verificacion	16 pruebas automáticas sin frameworks.

El dato principal es el `record Registro`. Un *record* es una clase especial de Java cuyos campos son `final`: después de crear un registro no hay forma de cambiarlo (OpenJDK, 2021). El constructor compacto además impide crear registros con datos vacíos:

```
/**
 * Un registro del sistema de transporte (lo que envia un validador o sensor).
 *
 * Es un "record": Java crea automaticamente el constructor, los metodos de
 * lectura, equals, hashCode y toString. Todos sus campos son final, por eso
 * un Registro NO se puede modificar despues de creado (es inmutable).
 */
public record Registro(String idUsuario,
                       String ruta,
```

```
                        String estacion,
                        Accion accion,
                        LocalDateTime timestamp) {

    /** Constructor compacto: impide crear registros con datos nulos. */
    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }

    /** Hora del dia (0 a 23) en que ocurrio el registro. */
    public int hora() {
        return timestamp.getHour();
    }

    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }
}
```

Generación de Datos Simulados

Como no se tienen los datos reales de TecnoValle, la clase `GeneradorDatos` simula un día completo con 8 rutas y 23 estaciones. Por defecto genera 150.000 usuarios, cada uno con 1 a 3 viajes, para un total de **509.644 registros**. Las horas tienen más peso en la mañana (6:00 a 8:59) y en la tarde (17:00 a 19:59), y las rutas troncales son más populares que las alimentadoras, para que los resultados se parezcan a la realidad.

Un detalle importante: el generador no usa `Random`, porque `Random` guarda un estado interno que cambia cada vez que se usa, y eso sería un efecto secundario. En su lugar, cada valor “al azar” se calcula con una función pura que mezcla la semilla, el número del usuario y un número de “sal”. Por eso, con la misma semilla siempre salen los mismos datos y la generación se puede hacer en paralelo:

```
/**
 * Funcion pura que "revuelve" tres numeros y devuelve uno que parece aleatorio
 * (algoritmo SplitMix64). Mismas entradas -> misma salida, siempre.
 */
static long mezclar(long semilla, long indice, long sal) {
    long z = semilla + indice * 0x9E3779B97F4A7C15L + sal * 0xD1B54A32D192ED03L;
    z = (z ^ (z >>> 30)) * 0xBF58476D1CE4E5B9L;
    z = (z ^ (z >>> 27)) * 0x94D049BB133111EBL;
    return z ^ (z >>> 31);
}

/**
 * Genera los registros de un dia completo, ordenados por hora.
```

```
 *
 * @param cantidadUsuarios cuantos usuarios distintos viajan
 * @param semilla          numero que define la simulacion
 * @return lista INMUTABLE de registros
 */
public static List<Registro> generar(int cantidadUsuarios, long semilla) {
    return IntStream.range(0, cantidadUsuarios)
            .parallel()
            .boxed()
            .flatMap(u -> registrosDeUsuario(u, semilla))
            .sorted(Comparator.comparing(Registro::timestamp)
                    .thenComparing(Registro::idUsuario)
                    .thenComparing(Registro::accion))
            .toList(); // toList() devuelve una lista que no se puede modificar
}
```

Implementación de las Tareas a) a f)

Todos los métodos reciben la lista de registros y un valor `paralelo`. Si es `true` usan `parallelStream()`; si es `false`, `stream()`. Este pequeño método decide cuál usar:

```
/** Crea el stream secuencial o paralelo segun se pida. */
private static Stream<Registro> flujo(List<Registro> registros, boolean paralelo) {
    return paralelo ? registros.parallelStream() : registros.stream();
}
```

a) Afluencia por Estación

Se filtran solo las entradas y se agrupan por estación contando cuántas hay. `groupingBy` hace el trabajo que antes hacía un ciclo con un mapa y un contador, y `collectingAndThen` envuelve el resultado para que no se pueda modificar (Oracle, s.f.-b).

```
/** Cuantas entradas hubo en cada estacion durante el dia. */
public static Map<String, Long> afluenciaPorEstacion(List<Registro> registros, boolean
paralelo) {
    return flujo(registros, paralelo)
            .filter(Filtros.ES_ENTRADA)
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(Registro::estacion, TreeMap::new,
Collectors.counting()),
                    Collections::unmodifiableMap));
}
```

Resultado: la estación con más afluencia es **Estacion Central**, con 29.115 entradas de 27.415 usuarios distintos, seguida de Plaza Mayor (22.726) y Portal Norte (17.084).

b) Horas Pico

La hora se obtiene con la función compuesta `HORA_DEL_REGISTRO` (ver más adelante) y luego se cuenta cuántos registros hay en cada hora. `horasPico` reutiliza la función genérica `topN` para escoger las horas con más flujo.

```
/** Flujo total (entradas + salidas) agrupado por hora del dia. */
public static Map<Integer, Long> flujoPorHora(List<Registro> registros, boolean paralelo) {
    return flujo(registros, paralelo)
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(Filtros.HORA_DEL_REGISTRO, TreeMap::new,
Collectors.counting()),
                    Collections::unmodifiableMap));
}

/**
 * Las n parejas (llave, cantidad) con mayor cantidad. Si hay empate, ordena por la llave.
 * Es generica: sirve para estaciones, horas, rutas, etc.
```

```
 */
public static <K extends Comparable<? super K>> List<Map.Entry<K, Long>> topN(Map<K, Long>
conteos, int n) {
    return conteos.entrySet().stream()
            .sorted(Map.Entry.<K, Long>comparingByValue().reversed()
                    .thenComparing(Map.Entry.<K, Long>comparingByKey()))
            .limit(n)
            .map(e -> Map.entry(e.getKey(), e.getValue()))
            .toList();
}
```

Resultado: las tres horas pico son **18:00–18:59** (47.173 registros), 17:00–17:59 (42.355) y 19:00–19:59 (36.526). En la mañana, la hora con más movimiento es 7:00–7:59 (33.781).

c) Rutas Más Utilizadas

```
/** Usos (entradas) de cada ruta. */
public static Map<String, Long> usosPorRuta(List<Registro> registros, boolean paralelo) {
    return flujo(registros, paralelo)
            .filter(Filtros.ES_ENTRADA)
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(Registro::ruta, TreeMap::new,
Collectors.counting()),
                    Collections::unmodifiableMap));
}
```

Resultado: la ruta más usada es **T1-Troncal Norte** con 53.821 usos, seguida de T2-Troncal Sur (45.471) y T3-Avenida Oriental (37.450). La menos usada es E1-Expreso Aeropuerto (13.508).

d) Patrones de Viaje por Usuario

`mapping` transforma cada registro en su estación y `toUnmodifiableList` las guarda en una lista que no se puede cambiar. Como la lista de origen está ordenada por hora, `groupingBy` conserva ese orden incluso en paralelo (Oracle, s.f.-a).

```
/**
 * Para cada usuario, la lista de estaciones que visito, en el mismo orden
 * en que aparecen en los registros. groupingBy conserva ese orden aunque
 * el stream sea paralelo, porque la lista de origen esta ordenada.
 */
public static Map<String, List<String>> patronesPorUsuario(List<Registro> registros, boolean
paralelo) {
    return flujo(registros, paralelo)
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(Registro::idUsuario, TreeMap::new,
                            Collectors.mapping(Registro::estacion,
Collectors.toUnmodifiableList())),
                    Collections::unmodifiableMap));
}
```

Resultado: se obtuvieron los patrones de los 150.000 usuarios. Por ejemplo, U000002: Plaza Mayor → Portal Oriente → Biblioteca → Estacion Central.

e) Tiempo Promedio entre Estaciones Consecutivas

Se agrupan los registros por usuario. Para cada usuario se crea una lista nueva ordenada por hora y, con `IntStream.range`, se arman los pares consecutivos (registro *i* con registro *i*+1). Un predicado de dos parámetros (`BiPredicate`) deja solo los pares que son un desplazamiento real: una entrada seguida de una salida en otra estación. Al final se calcula el promedio general y el promedio de cada tramo.

```
/** Un desplazamiento: va de una ENTRADA a la SALIDA siguiente del mismo usuario, en otra
estacion. */
private static final BiPredicate<Registro, Registro> ES_DESPLAZAMIENTO =
        (a, b) -> a.esEntrada() && !b.esEntrada() && !a.estacion().equals(b.estacion());

/** Convierte la lista de registros de UN usuario en sus desplazamientos. */
private static Stream<Desplazamiento> desplazamientosDe(List<Registro> registrosDelUsuario)
{
    List<Registro> ordenados = registrosDelUsuario.stream()
            .sorted(Comparator.comparing(Registro::timestamp))
            .toList();                                    // lista nueva, no se toca la
original
    return IntStream.range(0, ordenados.size() - 1)
            .mapToObj(i -> List.of(ordenados.get(i), ordenados.get(i + 1)))  // pares
consecutivos
            .filter(par -> ES_DESPLAZAMIENTO.test(par.get(0), par.get(1)))
            .map(par -> new Desplazamiento(
                    new Tramo(par.get(0).estacion(), par.get(1).estacion()),
                    minutosEntre(par.get(0), par.get(1))));
}

/** Tiempo promedio general y por tramo entre estaciones consecutivas. */
public static ResultadoTiempos tiemposEntreEstaciones(List<Registro> registros, boolean
paralelo) {
    List<Desplazamiento> desplazamientos = flujo(registros, paralelo)
            .collect(Collectors.groupingBy(Registro::idUsuario, TreeMap::new,
Collectors.toList()))
            .values()                                     // TreeMap: mismo orden siempre
            .stream()
            .flatMap(ProcesadorTransporte::desplazamientosDe)
            .toList();

    double promedioGeneral = desplazamientos.stream()
            .mapToDouble(Desplazamiento::minutos)
            .average()
            .orElse(0.0);

    Map<Tramo, Double> porTramo = desplazamientos.stream()
            .collect(Collectors.groupingBy(Desplazamiento::tramo,
                    Collectors.averagingDouble(Desplazamiento::minutos)));

    Map<Tramo, Long> viajesPorTramo = desplazamientos.stream()
            .collect(Collectors.groupingBy(Desplazamiento::tramo, Collectors.counting()));
```

```
    return new ResultadoTiempos(promedioGeneral, desplazamientos.size(), porTramo,
viajesPorTramo);
}
```

Resultado: se analizaron **254.190 desplazamientos** y el tiempo promedio entre estaciones consecutivas es de **10,0 minutos**. El tramo más recorrido es Los Pinos → Portal Norte (4.122 viajes, 10,5 minutos en promedio).

f) Detección de Sobrecarga en Rutas

Se agrupan las entradas por ruta y, dentro de cada ruta, por hora. Una función (`aEstado`) convierte cada ruta en un `EstadoRuta` con su hora pico y su ocupación. El predicado `esCritica` lo crea la función de orden superior `Filtros.superaUmbral(umbral)`.

```
/**
 * Calcula la ocupacion de cada ruta en su hora con mas entradas y la marca
 * como CRITICA si supera el umbral.
 *
 * @param capacidadPorHora pasajeros que la ruta puede mover en una hora (simulado)
 * @param umbral           por ejemplo 0.85 = 85 % de la capacidad
 */
public static List<EstadoRuta> estadoDeRutas(List<Registro> registros, long
capacidadPorHora,
                                             double umbral, boolean paralelo) {
    DoublePredicate esCritica = Filtros.superaUmbral(umbral);

    Map<String, Map<Integer, Long>> entradasPorRutaYHora = flujo(registros, paralelo)
            .filter(Filtros.ES_ENTRADA)
            .collect(Collectors.groupingBy(Registro::ruta,
                    Collectors.groupingBy(Registro::hora, Collectors.counting())));

    Function<Map.Entry<String, Map<Integer, Long>>, EstadoRuta> aEstado = e -> {
        Map.Entry<Integer, Long> pico = topN(e.getValue(), 1).get(0);
        long total = e.getValue().values().stream().mapToLong(Long::longValue).sum();
        double ocupacion = (double) pico.getValue() / capacidadPorHora;
        return new EstadoRuta(e.getKey(), total, pico.getKey(), pico.getValue(),
                ocupacion, esCritica.test(ocupacion));
    };

    return entradasPorRutaYHora.entrySet().stream()
            .map(aEstado)
            .sorted(Comparator.comparingDouble(EstadoRuta::ocupacion).reversed()
                    .thenComparing(EstadoRuta::ruta))
            .toList();
}
```

Resultado: con una capacidad simulada de 4.500 pasajeros por hora y un umbral del 85 %, dos rutas quedan **CRÍTICAS**: T1-Troncal Norte (112,5 % de ocupación entre las 18:00 y las 18:59) y T2-Troncal Sur (95,4 %). Las demás están en estado normal.

Conceptos de Programación Funcional Aplicados

Funciones Puras

Una función pura cumple dos reglas: (1) con los mismos datos de entrada siempre devuelve el mismo resultado y (2) no cambia nada por fuera de ella; es decir, no tiene *efectos secundarios* como imprimir, modificar variables globales o cambiar la lista que recibe (Urma et al., 2018). Es como una calculadora: 2 + 3 siempre da 5 y la calculadora no borra los números que le diste.

En el proyecto son puras todas las funciones de `ProcesadorTransporte`, `Filtros`, `GeneradorDatos` y `Reporte`. Por ejemplo, `Reporte.barra` no imprime la barra: la **devuelve** como texto, y quien la imprime es `App`:

```
/** Barra de texto proporcional al valor maximo. */
public static String barra(long valor, long maximo) {
    int largo = maximo == 0 ? 0 : (int) Math.round((double) valor / maximo * ANCHO_BARRA);
    return "#".repeat(largo);
}
```

Gracias a esto, la misma función se puede llamar las veces que se quiera y en paralelo sin que los resultados cambien, que es justo lo que fallaba en el sistema antiguo por las condiciones de carrera.

Inmutabilidad

Un objeto inmutable no se puede cambiar después de creado. Si se necesita otro valor, se crea un objeto nuevo (Bloch, 2018). En el proyecto se usó así:

- **Records:** `Registro`, `Tramo`, `EstadoRuta` y `ResultadoTiempos` tienen todos sus campos `final`.
- **Listas que no se pueden modificar:** `List.of(...)` y `Stream.toList()`. La lista de 509.644 registros es de este tipo: si alguien intenta agregarle algo, Java lanza `UnsupportedOperationException` (Figura 3).
- **Mapas que no se pueden modificar:** `Collections.unmodifiableMap`, `Map.copyOf` y `Collectors.toUnmodifiableList` en todos los resultados.
- **Nunca se ordena la lista original:** cuando se necesita otro orden se usa `sorted()`, que crea una lista nueva.

Al final del programa se compara la “huella” (`hashCode`) de la lista antes y después de todo el proceso, y es la misma: la lista original no cambió.

Lambdas y Referencias a Métodos

Una lambda es una función sin nombre que se escribe en una sola línea, con la forma `parámetros -> resultado`. Por ejemplo, `r -> r.accion() == Accion.ENTRADA` recibe un registro y responde si es una entrada. Una *referencia a método*, como `Registro::estacion`, es una forma más corta de escribir `r -> r.estacion()`. El proyecto las usa en todos los `filter`, `map`, `groupingBy` y `sorted`.

Funciones de Orden Superior y Composición

Una función de orden superior es una función que **recibe** otra función como parámetro o que **devuelve** una función como resultado (Urma et al., 2018). En la clase `Filtros` hay varias que devuelven funciones nuevas. Por ejemplo, `entreHoras(6, 9)` fabrica un predicado que dice si un registro ocurrió entre las 6:00 y las 8:59, y `superaUmbral(0.85)` fabrica uno que dice si una ocupación pasa del 85 %. Además, las funciones se **componen** como piezas de LEGO: `negate()` invierte un predicado, `and()` une dos y `andThen()` encadena dos funciones:

```
/** true si el registro es una ENTRADA. */
public static final Predicate<Registro> ES_ENTRADA = r -> r.accion() == Accion.ENTRADA;

/** true si el registro es una SALIDA (se construye negando el anterior). */
public static final Predicate<Registro> ES_SALIDA = ES_ENTRADA.negate();

/** Composicion de funciones: Registro -> fecha y hora -> hora del dia. */
public static final Function<Registro, Integer> HORA_DEL_REGISTRO =
        ((Function<Registro, LocalDateTime>)
Registro::timestamp).andThen(LocalDateTime::getHour);

/**
 * Funcion de orden superior: RECIBE dos horas y DEVUELVE un predicado nuevo.
 * Ejemplo: entreHoras(6, 9) es "el registro ocurrio entre las 6:00 y las 8:59".
 */
public static Predicate<Registro> entreHoras(int desde, int hastaSinIncluir) {
    return r -> HORA_DEL_REGISTRO.apply(r) >= desde && HORA_DEL_REGISTRO.apply(r) <
hastaSinIncluir;
}

/** Funcion de orden superior: devuelve un predicado que revisa la ruta. */
public static Predicate<Registro> deRuta(String ruta) {
    return r -> r.ruta().equals(ruta);
}

/** Funcion de orden superior: devuelve un predicado que dice si una ocupacion supera el
umbral. */
```

```
public static DoublePredicate superaUmbral(double umbral) {
    return ocupacion -> ocupacion > umbral;
}

/** Ejemplo de composicion: entradas en la hora pico de la manana (6:00 a 8:59). */
public static final Predicate<Registro> ENTRADA_PICO_MANANA = ES_ENTRADA.and(entreHoras(6,
9));
```

También reciben funciones `topN` (usa comparadores), `Reporte.tablaConBarras` (recibe una `Function` que decide cómo mostrar cada llave) y `App.medir` (recibe un `Supplier` con la operación que se va a cronometrar).

Streams

Un *Stream* es una “tubería” por la que pasan los datos y en la que cada etapa hace una sola cosa: filtrar, transformar, agrupar, ordenar o resumir (Oracle, s.f.-a). A diferencia de un ciclo `for`, el stream no dice cómo recorrer la lista ni guarda contadores: solo describe el resultado que se quiere, por eso se dice que es *declarativo*. La Tabla 3 resume las operaciones usadas.

Tabla 3

Operaciones de Streams usadas en el proyecto

Operación	**Qué hace**	**Dónde se usa**
`filter`	Deja pasar solo los elementos que cumplen un predicado	Tareas a, c, f
`map` / `mapToObj`	Transforma cada elemento en otro	Tareas e, f; Reporte
`flatMap`	Convierte cada elemento en varios y los junta en un solo stream	GeneradorDatos; tarea e
`sorted` / `limit`	Ordena (creando un stream nuevo) y toma los primeros	topN; tarea e; generador
`groupingBy` + `counting`	Agrupa por una llave y cuenta	Tareas a, b, c, f
`mapping` + `toUnmodifiableList`	Agrupa guardando solo un dato, en lista inmutable	Tarea d
`averagingDouble` / `average`	Calcula promedios	Tarea e
`collectingAndThen`	Aplica una función al resultado final (hacerlo inmutable)	Tareas a, b, c, d
`IntStream.range`	Genera los números de 0 a n−1 sin un ciclo	Generador; pares de la tarea e
`parallelStream`	Divide el trabajo entre los núcleos del procesador	Todas las tareas

Paralelismo

Como ninguna función modifica datos compartidos, se pueden ejecutar en paralelo con `parallelStream()` sin condiciones de carrera. El programa ejecuta las seis tareas primero en modo secuencial y luego en paralelo, y compara los dos resultados con `equals`: son **exactamente iguales**. En la prueba, con un procesador de 2 núcleos, el modo secuencial tardó 1.831 ms y el paralelo 1.222 ms, es decir, cerca de un 33 % menos (Figura 5). Con más núcleos la diferencia debería ser mayor.

Pruebas del Código

Como la actividad pide no usar frameworks externos (como JUnit), se creó la clase `Verificacion`. Esta clase usa 10 registros hechos a mano, cuyos resultados se calcularon con lápiz y papel, y los compara con lo que devuelven las funciones. Hay 16 pruebas: dos o más por cada tarea, una de predicados compuestos, una que compara paralelo contra secuencial y dos de inmutabilidad. **Las 16 pruebas pasaron** (Figura 2).

```
/** Datos de prueba: 3 usuarios, 2 rutas, 10 registros. */
private static final List<Registro> DATOS = List.of(
        new Registro("U1", "R1", "A", Accion.ENTRADA, t(7, 0)),
        new Registro("U2", "R1", "A", Accion.ENTRADA, t(7, 10)),
        new Registro("U1", "R1", "C", Accion.SALIDA, t(7, 20)),
        new Registro("U3", "R2", "B", Accion.ENTRADA, t(7, 30)),
        new Registro("U2", "R1", "B", Accion.SALIDA, t(7, 40)),
        new Registro("U3", "R2", "D", Accion.SALIDA, t(8, 0)),
        new Registro("U1", "R2", "C", Accion.ENTRADA, t(17, 0)),
        new Registro("U1", "R2", "D", Accion.SALIDA, t(17, 30)),
        new Registro("U2", "R1", "B", Accion.ENTRADA, t(18, 0)),
        new Registro("U2", "R1", "A", Accion.SALIDA, t(18, 10))
);
```

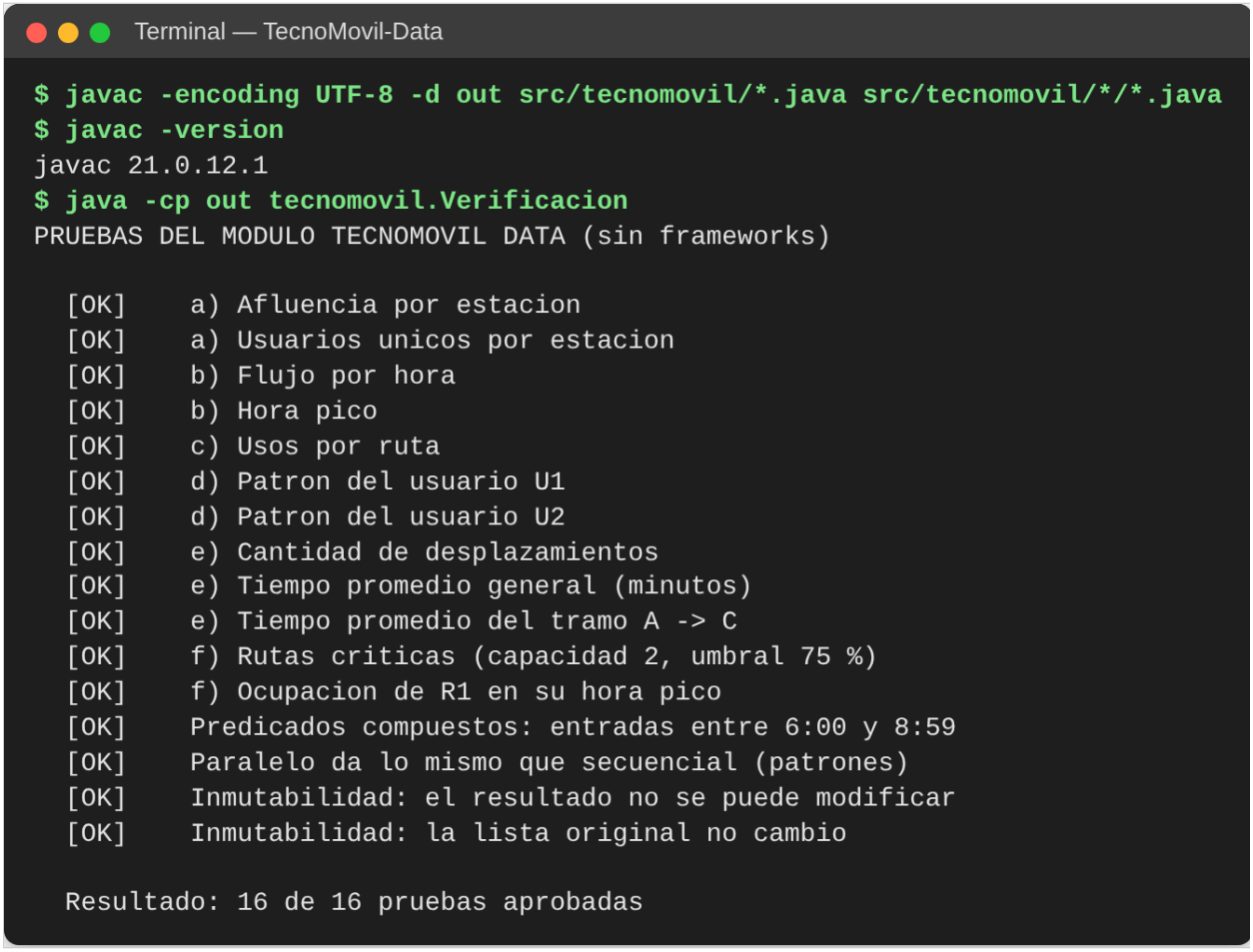
Por ejemplo, el tiempo promedio esperado es 24 minutos porque los cinco desplazamientos duran 20, 30, 30, 10 y 30 minutos, y (20 + 30 + 30 + 10 + 30) ÷ 5 = 24.

Evidencia de Ejecución

El proyecto se compiló con `javac` (Java 21) y se ejecutó con 150.000 usuarios. Las siguientes capturas muestran la salida del programa en la consola.

Figura 2

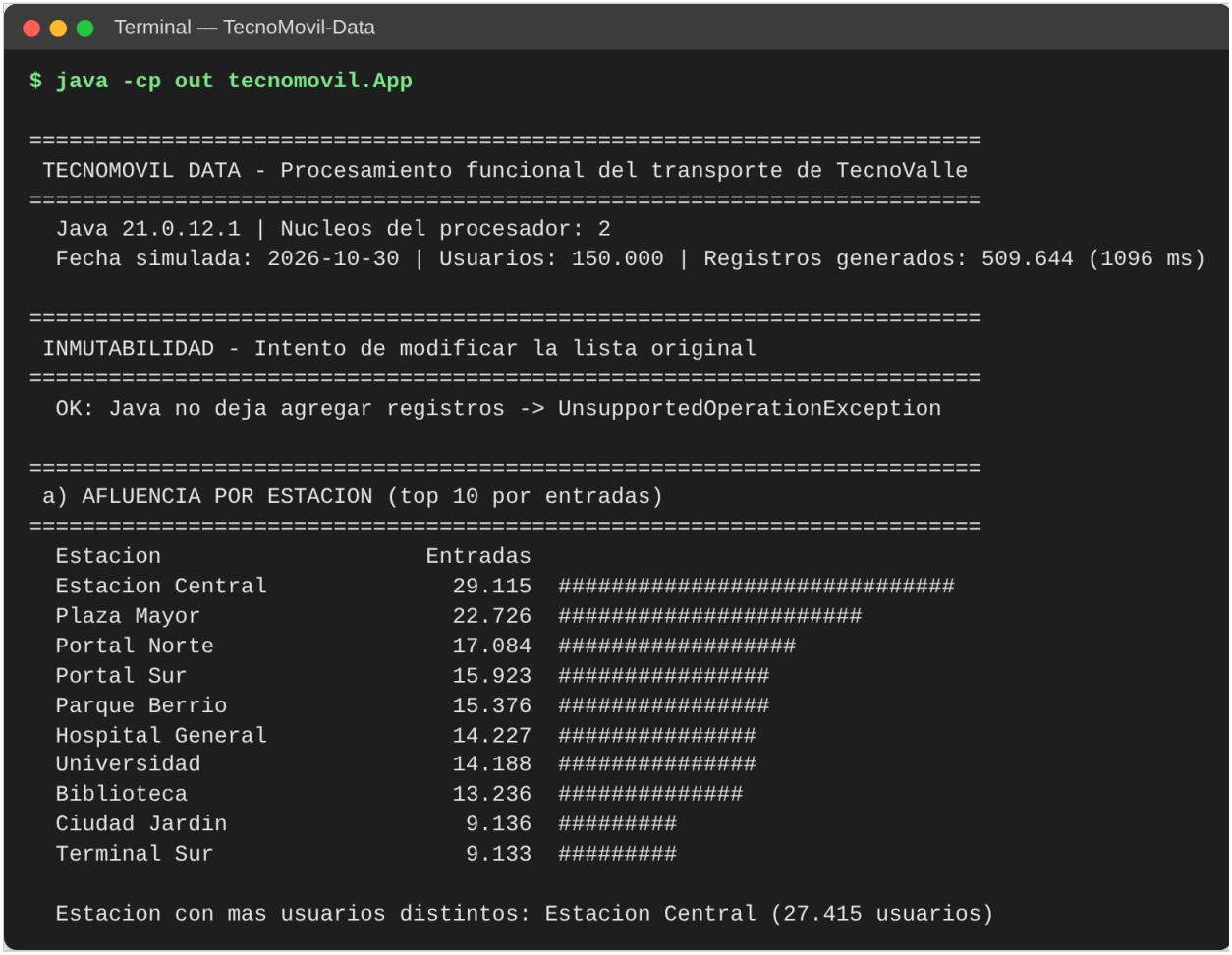
Compilación del proyecto y resultado de las 16 pruebas



Nota. Captura de la ejecución del programa en la consola.

Figura 3

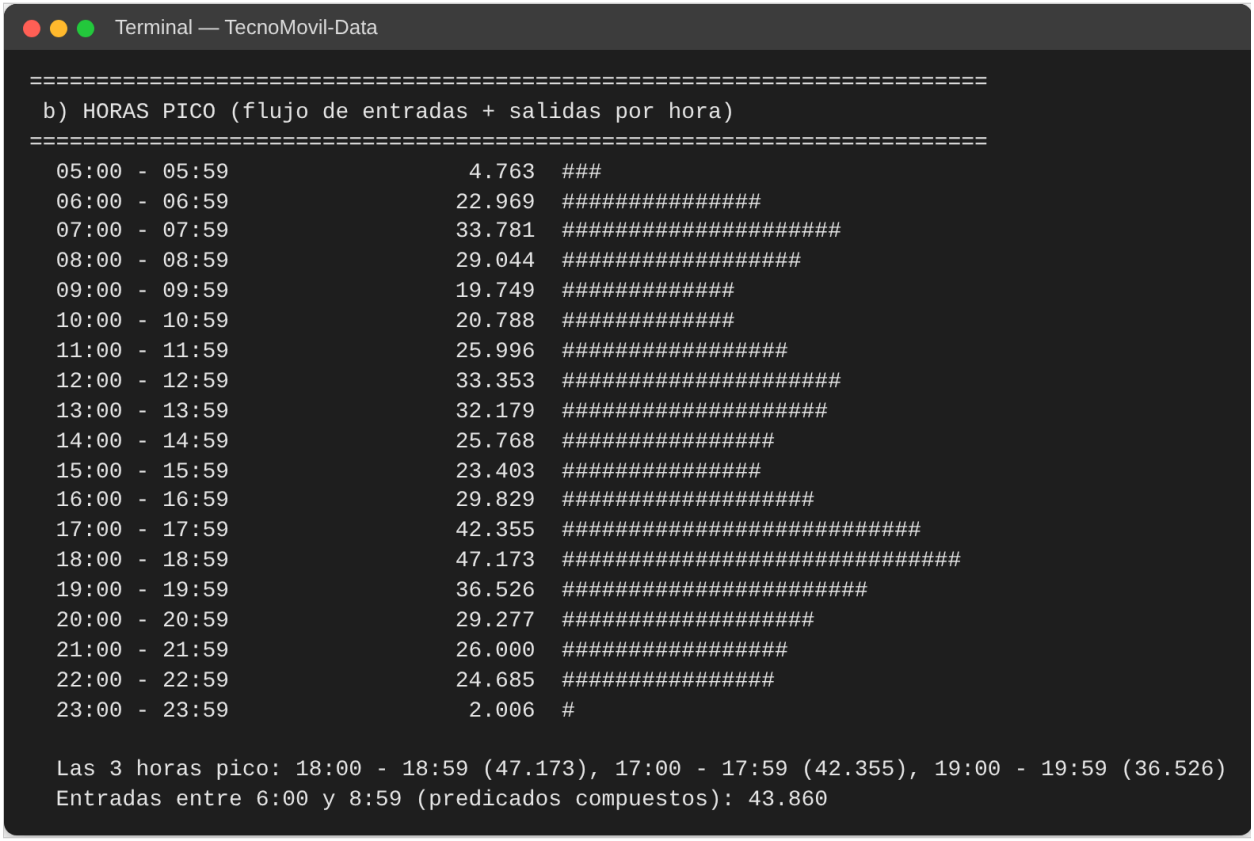
Inicio del programa, prueba de inmutabilidad y afluencia por estación (tarea a)



Nota. Captura de la ejecución del programa en la consola.

Figura 4

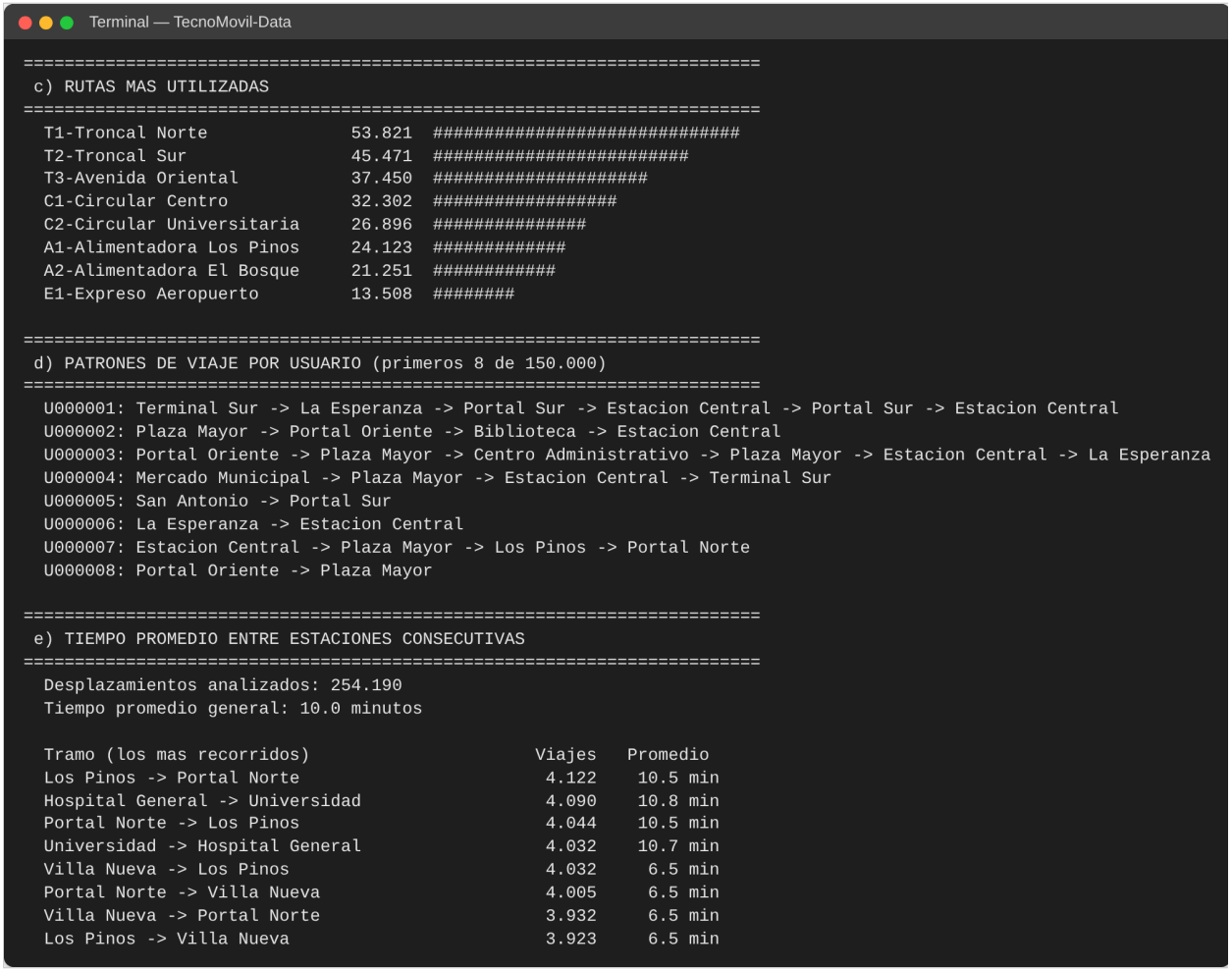
Flujo por hora y horas pico (tarea b)



Nota. Captura de la ejecución del programa en la consola.

Figura 5

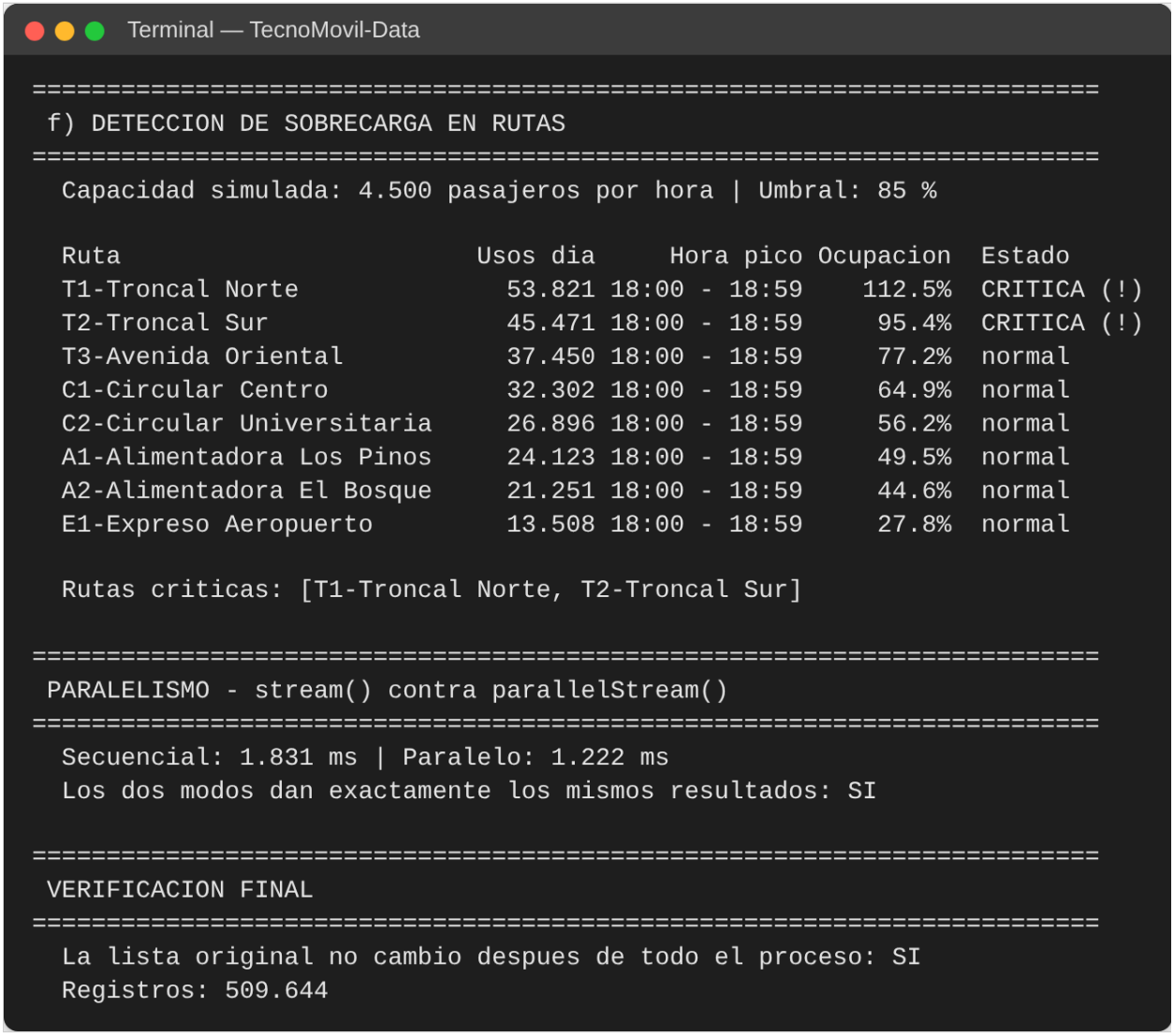
Rutas más utilizadas, patrones de viaje y tiempos entre estaciones (tareas c, d y e)



Nota. Captura de la ejecución del programa en la consola.

Figura 6

Sobrecarga en rutas, comparación secuencial contra paralelo y verificación final (tarea f)



Nota. Captura de la ejecución del programa en la consola.

Repositorio en GitHub y Archivo ZIP

El proyecto se publicó en un repositorio público de GitHub, en la rama principal `main`:

https://github.com/davidezapatar/TecnoMovil-Data

El repositorio contiene la carpeta `src` con el código fuente, los archivos `ejecutar.bat` (Windows) y `ejecutar.sh` (Linux/Mac) para compilar y ejecutar con un solo paso, el archivo `README.md` con la explicación del proyecto, el archivo `.gitignore` para no subir archivos compilados y la carpeta `docs` con este informe. Junto con este informe se entrega el archivo `TecnoMovil-Data.zip` con el proyecto final.

Para ejecutar el proyecto solo se necesita Java 17 o superior: se descarga el repositorio y se hace doble clic en `ejecutar.bat`, o se usan estos comandos:

```
javac -encoding UTF-8 -d out src/tecnomovil/*.java src/tecnomovil/*/*.java
java -cp out tecnomovil.Verificacion
java -cp out tecnomovil.App
java -cp out tecnomovil.App 300000      (para simular otra cantidad de usuarios)
```

Conclusiones

- Se cumplió el objetivo general: el módulo resuelve las seis tareas del caso de estudio usando programación funcional y está publicado en GitHub.
- Los Streams permitieron procesar 509.644 registros en menos de dos segundos con código corto y fácil de leer. Cada tarea se escribió como una “tubería” de pocas líneas, en lugar de ciclos largos llenos de contadores.
- Usar funciones puras e inmutabilidad elimina el problema de las condiciones de carrera del sistema antiguo. La prueba es que el modo paralelo dio exactamente los mismos resultados que el secuencial y fue cerca de un 33 % más rápido.
- Las funciones de orden superior y la composición (`and`, `negate`, `andThen`) permitieron crear piezas pequeñas y reutilizables. Por ejemplo, `topN` sirve igual para estaciones, horas y rutas.
- Separar el núcleo funcional (cálculos) del borde (imprimir en pantalla) facilitó las pruebas: las 16 pruebas de `Verificacion` se escribieron sin ningún framework.
- Con los datos simulados, TecnoValle debería prestar atención a las rutas T1-Troncal Norte y T2-Troncal Sur entre las 18:00 y las 19:00, porque superan el 85 % de su capacidad.

Enlaces del Repositorio

Tabla 4

Repositorio de cada integrante del grupo

Integrante	Usuario de GitHub	Enlace del repositorio
David Esteban Zapata Rodríguez	davidezapatar	https://github.com/davidezapatar/TecnoMovil-Data

Nota. El trabajo se realizó de forma individual en el Grupo 10.

Referencias

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Oracle. (s.f.-a). *Package java.util.stream*. Java SE 21 & JDK 21 API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html

Oracle. (s.f.-b). *Class Collectors*. Java SE 21 & JDK 21 API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/Collectors.html

OpenJDK. (2021). *JEP 395: Records*. https://openjdk.org/jeps/395

Urma, R.-G., Fusco, M., y Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming*. Manning.

Anexo. Código Fuente Completo

Este es el código fuente completo del proyecto, el mismo que está en el repositorio, ordenado por paquete.

src/tecnomovil/modelo/Accion.java

```
package tecnomovil.modelo;

/**
 * Tipo de validacion que hace un usuario en una estacion.
 * ENTRADA: el usuario ingresa a la estacion.
 * SALIDA: el usuario valida para salir o para cambiar de ruta.
 */
public enum Accion {
    ENTRADA,
    SALIDA
}
```

src/tecnomovil/modelo/Registro.java

```
package tecnomovil.modelo;

import java.time.LocalDateTime;
import java.util.Objects;

/**
 * Un registro del sistema de transporte (lo que envia un validador o sensor).
 *
 * Es un "record": Java crea automaticamente el constructor, los metodos de
 * lectura, equals, hashCode y toString. Todos sus campos son final, por eso
 * un Registro NO se puede modificar despues de creado (es inmutable).
 */
public record Registro(String idUsuario,
                       String ruta,
                       String estacion,
                       Accion accion,
                       LocalDateTime timestamp) {

    /** Constructor compacto: impide crear registros con datos nulos. */
    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }

    /** Hora del dia (0 a 23) en que ocurrio el registro. */
    public int hora() {
        return timestamp.getHour();
    }

    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }
}
```

src/tecnomovil/modelo/Tramo.java

```
package tecnomovil.modelo;

/**
 * Desplazamiento entre dos estaciones consecutivas (origen -> destino).
 * Se usa como llave para agrupar los tiempos de viaje.
 */
public record Tramo(String origen, String destino) {

    @Override
    public String toString() {
        return origen + " -> " + destino;
    }
}
```

src/tecnomovil/modelo/EstadoRuta.java

```
package tecnomovil.modelo;

/**
 * Resultado del analisis de ocupacion de una ruta.
 *
 * @param ruta            nombre de la ruta
 * @param usosTotales     entradas registradas en el dia
 * @param horaPico        hora con mas entradas en la ruta
 * @param entradasEnPico  entradas registradas en esa hora
 * @param ocupacion       entradasEnPico / capacidad por hora (1.0 = 100 %)
 * @param critica         true si la ocupacion supera el umbral definido
 */
public record EstadoRuta(String ruta,
                         long usosTotales,
                         int horaPico,
                         long entradasEnPico,
                         double ocupacion,
                         boolean critica) {

    public String estado() {
        return critica ? "CRITICA" : "NORMAL";
    }
}
```

src/tecnomovil/modelo/ResultadoTiempos.java

```
package tecnomovil.modelo;

import java.util.Map;

/**
 * Resultado del calculo de tiempos entre estaciones consecutivas.
 *
 * @param promedioGeneralMinutos tiempo promedio de todos los desplazamientos
 * @param totalDesplazamientos   cantidad de desplazamientos analizados
 * @param promedioPorTramo       tiempo promedio (minutos) de cada tramo origen -> destino
 * @param viajesPorTramo         cuantas veces se recorrio cada tramo
 */
public record ResultadoTiempos(double promedioGeneralMinutos,
                               long totalDesplazamientos,
                               Map<Tramo, Double> promedioPorTramo,
                               Map<Tramo, Long> viajesPorTramo) {

    public ResultadoTiempos {
        promedioPorTramo = Map.copyOf(promedioPorTramo); // copias inmutables
        viajesPorTramo = Map.copyOf(viajesPorTramo);
    }
}
```

src/tecnomovil/datos/GeneradorDatos.java

```
package tecnomovil.datos;

import tecnomovil.modelo.Accion;
import tecnomovil.modelo.Registro;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/**
 * Genera datos simulados de un dia de operacion del transporte de TecnoValle.
 *
 * Todo el generador es PURO: no usa Random con estado. Cada valor "aleatorio"
 * se calcula con una formula a partir de la semilla y del numero del usuario.
 * Por eso, con la misma semilla y la misma cantidad de usuarios, el resultado
 * es SIEMPRE el mismo, y se puede generar en paralelo sin problemas.
 */
public final class GeneradorDatos {

    private GeneradorDatos() { }

    /** Dia que se simula. */
    public static final LocalDate FECHA = LocalDate.of(2026, 10, 30);

    /** Definicion de una ruta: su nombre, sus estaciones en orden y su "popularidad". */
    public record Ruta(String nombre, List<String> estaciones, int peso) {
        public Ruta {
            estaciones = List.copyOf(estaciones);
        }
    }

    /** Las 8 rutas del sistema (datos ficticios). */
    public static final List<Ruta> RUTAS = List.of(
        new Ruta("T1-Troncal Norte", List.of("Portal Norte", "Calle 100", "Unicentro", "Estacion
Central",
                "Plaza Mayor", "Parque Berrio"), 20),
        new Ruta("T2-Troncal Sur", List.of("Portal Sur", "Ciudad Jardin", "La Esperanza", "Estacion
Central",
                "Terminal Sur"), 17),
        new Ruta("T3-Avenida Oriental", List.of("Portal Oriente", "Hospital General", "Universidad",
                "Plaza Mayor", "Centro Administrativo"), 14),
        new Ruta("A1-Alimentadora Los Pinos", List.of("Los Pinos", "Villa Nueva", "Portal Norte"), 9),
        new Ruta("A2-Alimentadora El Bosque", List.of("El Bosque", "San Antonio", "Portal Sur"), 8),
        new Ruta("C1-Circular Centro", List.of("Estacion Central", "Mercado Municipal", "Parque
Berrio",
                "Biblioteca", "Plaza Mayor"), 12),
        new Ruta("E1-Expreso Aeropuerto", List.of("Estacion Central", "Zona Franca", "Aeropuerto"),
5),
        new Ruta("C2-Circular Universitaria", List.of("Universidad", "Parque Tecnologico",
"Biblioteca",
                "Hospital General"), 10)
    );

    /** Peso de cada hora del dia (0 a 23). Mas peso = mas viajes empiezan a esa hora. */
    private static final int[] PESO_HORA = {
        0, 0, 0, 0, 0, 3, 14, 20, 16, 8, 6, 6, 9, 8, 6, 6, 10, 17, 19, 12, 7, 4, 2, 0
    };

    /** Lista de horas repetidas segun su peso (para elegir horas con mas probabilidad). */
    private static final List<Integer> HORAS_PONDERADAS = IntStream.range(0, 24)
            .boxed()
            .flatMap(h -> Collections.nCopies(PESO_HORA[h], h).stream())
```

```
            .toList();

    /** Lista de rutas repetidas segun su peso. */
    private static final List<Ruta> RUTAS_PONDERADAS = RUTAS.stream()
            .flatMap(r -> Collections.nCopies(r.peso(), r).stream())
            .toList();

    /**
     * Genera los registros de un dia completo, ordenados por hora.
     *
     * @param cantidadUsuarios cuantos usuarios distintos viajan
     * @param semilla          numero que define la simulacion
     * @return lista INMUTABLE de registros
     */
    public static List<Registro> generar(int cantidadUsuarios, long semilla) {
        return IntStream.range(0, cantidadUsuarios)
                .parallel()
                .boxed()
                .flatMap(u -> registrosDeUsuario(u, semilla))
                .sorted(Comparator.comparing(Registro::timestamp)
                        .thenComparing(Registro::idUsuario)
                        .thenComparing(Registro::accion))
                .toList(); // toList() devuelve una lista que no se puede modificar
    }

    /** Todos los registros de un usuario: 1 a 3 viajes, cada uno con una ENTRADA y una SALIDA. */
    private static Stream<Registro> registrosDeUsuario(int usuario, long semilla) {
        String id = String.format("U%06d", usuario + 1);
        int viajes = 1 + elegir(mezclar(semilla, usuario, 1), 3);
        int horaInicio = HORAS_PONDERADAS.get(elegir(mezclar(semilla, usuario, 2),
HORAS_PONDERADAS.size()));
        return IntStream.range(0, viajes)
                .boxed()
                .flatMap(viaje -> registrosDeViaje(id, usuario, viaje, horaInicio, semilla));
    }

    /** Los dos registros (entrada y salida) de un viaje. Devuelve un stream vacio si el viaje queda
despues de las 22 h. */
    private static Stream<Registro> registrosDeViaje(String id, int usuario, int viaje, int
horaInicio, long semilla) {
        long base = usuario * 10L + viaje;
        int hora = horaInicio + viaje * (3 + elegir(mezclar(semilla, base, 3), 4));
        if (hora > 22) {
            return Stream.empty();
        }
        Ruta ruta = RUTAS_PONDERADAS.get(elegir(mezclar(semilla, base, 4), RUTAS_PONDERADAS.size()));
        int n = ruta.estaciones().size();
        int origen = elegir(mezclar(semilla, base, 5), n);
        int destino = (origen + 1 + elegir(mezclar(semilla, base, 6), n - 1)) % n;   // siempre
distinto al origen
        int paradas = Math.abs(destino - origen);
        int minutosViaje = paradas * (3 + elegir(mezclar(semilla, base, 7), 3)) +
elegir(mezclar(semilla, base, 8), 5);

        LocalDateTime entrada = FECHA.atTime(hora, elegir(mezclar(semilla, base, 9), 60),
                elegir(mezclar(semilla, base, 10), 60));
        LocalDateTime salida = entrada.plusMinutes(minutosViaje).plusSeconds(elegir(mezclar(semilla,
base, 11), 60));

        return Stream.of(
                new Registro(id, ruta.nombre(), ruta.estaciones().get(origen), Accion.ENTRADA,
entrada),
                new Registro(id, ruta.nombre(), ruta.estaciones().get(destino), Accion.SALIDA,
salida));
    }

    /**
     * Funcion pura que "revuelve" tres numeros y devuelve uno que parece aleatorio
     * (algoritmo SplitMix64). Mismas entradas -> misma salida, siempre.
     */
```

```
    static long mezclar(long semilla, long indice, long sal) {
        long z = semilla + indice * 0x9E3779B97F4A7C15L + sal * 0xD1B54A32D192ED03L;
        z = (z ^ (z >>> 30)) * 0xBF58476D1CE4E5B9L;
        z = (z ^ (z >>> 27)) * 0x94D049BB133111EBL;
        return z ^ (z >>> 31);
    }

    /** Convierte un numero revuelto en una posicion entre 0 y n-1. */
    static int elegir(long numero, int n) {
        return (int) Math.floorMod(numero, (long) n);
    }
}
```

src/tecnomovil/procesamiento/Filtros.java

```
package tecnomovil.procesamiento;

import tecnomovil.modelo.Accion;
import tecnomovil.modelo.Registro;

import java.time.LocalDateTime;
import java.util.function.DoublePredicate;
import java.util.function.Function;
import java.util.function.Predicate;

/**
 * Funciones pequenas y reutilizables (predicados y funciones) que se
 * combinan entre si con and(), or(), negate() y andThen().
 */
public final class Filtros {

    private Filtros() { }

    /** true si el registro es una ENTRADA. */
    public static final Predicate<Registro> ES_ENTRADA = r -> r.accion() == Accion.ENTRADA;

    /** true si el registro es una SALIDA (se construye negando el anterior). */
    public static final Predicate<Registro> ES_SALIDA = ES_ENTRADA.negate();

    /** Composicion de funciones: Registro -> fecha y hora -> hora del dia. */
    public static final Function<Registro, Integer> HORA_DEL_REGISTRO =
            ((Function<Registro, LocalDateTime>) Registro::timestamp).andThen(LocalDateTime::getHour);

    /**
     * Funcion de orden superior: RECIBE dos horas y DEVUELVE un predicado nuevo.
     * Ejemplo: entreHoras(6, 9) es "el registro ocurrio entre las 6:00 y las 8:59".
     */
    public static Predicate<Registro> entreHoras(int desde, int hastaSinIncluir) {
        return r -> HORA_DEL_REGISTRO.apply(r) >= desde && HORA_DEL_REGISTRO.apply(r) <
hastaSinIncluir;
    }

    /** Funcion de orden superior: devuelve un predicado que revisa la ruta. */
    public static Predicate<Registro> deRuta(String ruta) {
        return r -> r.ruta().equals(ruta);
    }

    /** Funcion de orden superior: devuelve un predicado que dice si una ocupacion supera el umbral.
*/
    public static DoublePredicate superaUmbral(double umbral) {
        return ocupacion -> ocupacion > umbral;
    }

    /** Ejemplo de composicion: entradas en la hora pico de la manana (6:00 a 8:59). */
    public static final Predicate<Registro> ENTRADA_PICO_MANANA = ES_ENTRADA.and(entreHoras(6, 9));
}
```

src/tecnomovil/procesamiento/ProcesadorTransporte.java

```
package tecnomovil.procesamiento;

import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.Registro;
import tecnomovil.modelo.ResultadoTiempos;
import tecnomovil.modelo.Tramo;

import java.time.Duration;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.BiPredicate;
import java.util.function.DoublePredicate;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/**
 * Modulo de procesamiento funcional de TecnoMovil Data.
 *
 * Reglas que cumplen TODOS los metodos de esta clase:
 *  1. Son funciones puras: el resultado depende solo de los parametros y no
 *     modifican nada fuera de ellas (no imprimen, no cambian variables globales).
 *  2. Nunca modifican la lista de registros que reciben.
 *  3. Devuelven colecciones que no se pueden modificar.
 *  4. Con paralelo = true usan parallelStream() y el resultado es el mismo.
 */
public final class ProcesadorTransporte {

    private ProcesadorTransporte() { }

    /** Crea el stream secuencial o paralelo segun se pida. */
    private static Stream<Registro> flujo(List<Registro> registros, boolean paralelo) {
        return paralelo ? registros.parallelStream() : registros.stream();
    }

    // ------------------------------------------------------------------
    // a) Afluencia por estacion
    // ------------------------------------------------------------------

    /** Cuantas entradas hubo en cada estacion durante el dia. */
    public static Map<String, Long> afluenciaPorEstacion(List<Registro> registros, boolean paralelo) {
        return flujo(registros, paralelo)
                .filter(Filtros.ES_ENTRADA)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::estacion, TreeMap::new,
Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    /** Cuantos usuarios DISTINTOS entraron a cada estacion (un usuario cuenta una sola vez). */
    public static Map<String, Long> usuariosUnicosPorEstacion(List<Registro> registros, boolean
paralelo) {
        return flujo(registros, paralelo)
                .filter(Filtros.ES_ENTRADA)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::estacion, TreeMap::new,
                                Collectors.collectingAndThen(
                                        Collectors.mapping(Registro::idUsuario, Collectors.toSet()),
                                        usuarios -> (long) usuarios.size())),
                        Collections::unmodifiableMap));
    }
```

```
    // ------------------------------------------------------------------
    // b) Horas pico
    // ------------------------------------------------------------------

    /** Flujo total (entradas + salidas) agrupado por hora del dia. */
    public static Map<Integer, Long> flujoPorHora(List<Registro> registros, boolean paralelo) {
        return flujo(registros, paralelo)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Filtros.HORA_DEL_REGISTRO, TreeMap::new,
Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    /** Las n horas con mas flujo. */
    public static List<Map.Entry<Integer, Long>> horasPico(Map<Integer, Long> flujoPorHora, int n) {
        return topN(flujoPorHora, n);
    }

    // ------------------------------------------------------------------
    // c) Rutas mas utilizadas
    // ------------------------------------------------------------------

    /** Usos (entradas) de cada ruta. */
    public static Map<String, Long> usosPorRuta(List<Registro> registros, boolean paralelo) {
        return flujo(registros, paralelo)
                .filter(Filtros.ES_ENTRADA)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::ruta, TreeMap::new, Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    // ------------------------------------------------------------------
    // d) Patrones de viaje por usuario
    // ------------------------------------------------------------------

    /**
     * Para cada usuario, la lista de estaciones que visito, en el mismo orden
     * en que aparecen en los registros. groupingBy conserva ese orden aunque
     * el stream sea paralelo, porque la lista de origen esta ordenada.
     */
    public static Map<String, List<String>> patronesPorUsuario(List<Registro> registros, boolean
paralelo) {
        return flujo(registros, paralelo)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::idUsuario, TreeMap::new,
                                Collectors.mapping(Registro::estacion,
Collectors.toUnmodifiableList())),
                        Collections::unmodifiableMap));
    }

    // ------------------------------------------------------------------
    // e) Tiempo promedio entre estaciones consecutivas
    // ------------------------------------------------------------------

    /** Un desplazamiento: va de una ENTRADA a la SALIDA siguiente del mismo usuario, en otra
estacion. */
    private static final BiPredicate<Registro, Registro> ES_DESPLAZAMIENTO =
            (a, b) -> a.esEntrada() && !b.esEntrada() && !a.estacion().equals(b.estacion());

    /** Minutos entre dos registros (con decimales). */
    private static double minutosEntre(Registro a, Registro b) {
        return Duration.between(a.timestamp(), b.timestamp()).toSeconds() / 60.0;
    }

    /** Par (tramo, minutos) de un desplazamiento. */
    private record Desplazamiento(Tramo tramo, double minutos) { }

    /** Convierte la lista de registros de UN usuario en sus desplazamientos. */
    private static Stream<Desplazamiento> desplazamientosDe(List<Registro> registrosDelUsuario) {
        List<Registro> ordenados = registrosDelUsuario.stream()
```

```
                .sorted(Comparator.comparing(Registro::timestamp))
                .toList();                                      // lista nueva, no se toca la original
        return IntStream.range(0, ordenados.size() - 1)
                .mapToObj(i -> List.of(ordenados.get(i), ordenados.get(i + 1)))  // pares consecutivos
                .filter(par -> ES_DESPLAZAMIENTO.test(par.get(0), par.get(1)))
                .map(par -> new Desplazamiento(
                        new Tramo(par.get(0).estacion(), par.get(1).estacion()),
                        minutosEntre(par.get(0), par.get(1))));
    }

    /** Tiempo promedio general y por tramo entre estaciones consecutivas. */
    public static ResultadoTiempos tiemposEntreEstaciones(List<Registro> registros, boolean paralelo)
{
        List<Desplazamiento> desplazamientos = flujo(registros, paralelo)
                .collect(Collectors.groupingBy(Registro::idUsuario, TreeMap::new,
Collectors.toList()))
                .values()                                       // TreeMap: mismo orden siempre
                .stream()
                .flatMap(ProcesadorTransporte::desplazamientosDe)
                .toList();

        double promedioGeneral = desplazamientos.stream()
                .mapToDouble(Desplazamiento::minutos)
                .average()
                .orElse(0.0);

        Map<Tramo, Double> porTramo = desplazamientos.stream()
                .collect(Collectors.groupingBy(Desplazamiento::tramo,
                        Collectors.averagingDouble(Desplazamiento::minutos)));

        Map<Tramo, Long> viajesPorTramo = desplazamientos.stream()
                .collect(Collectors.groupingBy(Desplazamiento::tramo, Collectors.counting()));

        return new ResultadoTiempos(promedioGeneral, desplazamientos.size(), porTramo,
viajesPorTramo);
    }

    // ------------------------------------------------------------------
    // f) Deteccion de sobrecarga en rutas
    // ------------------------------------------------------------------

    /**
     * Calcula la ocupacion de cada ruta en su hora con mas entradas y la marca
     * como CRITICA si supera el umbral.
     *
     * @param capacidadPorHora pasajeros que la ruta puede mover en una hora (simulado)
     * @param umbral           por ejemplo 0.85 = 85 % de la capacidad
     */
    public static List<EstadoRuta> estadoDeRutas(List<Registro> registros, long capacidadPorHora,
                                                 double umbral, boolean paralelo) {
        DoublePredicate esCritica = Filtros.superaUmbral(umbral);

        Map<String, Map<Integer, Long>> entradasPorRutaYHora = flujo(registros, paralelo)
                .filter(Filtros.ES_ENTRADA)
                .collect(Collectors.groupingBy(Registro::ruta,
                        Collectors.groupingBy(Registro::hora, Collectors.counting())));

        Function<Map.Entry<String, Map<Integer, Long>>, EstadoRuta> aEstado = e -> {
            Map.Entry<Integer, Long> pico = topN(e.getValue(), 1).get(0);
            long total = e.getValue().values().stream().mapToLong(Long::longValue).sum();
            double ocupacion = (double) pico.getValue() / capacidadPorHora;
            return new EstadoRuta(e.getKey(), total, pico.getKey(), pico.getValue(),
                    ocupacion, esCritica.test(ocupacion));
        };

        return entradasPorRutaYHora.entrySet().stream()
                .map(aEstado)
                .sorted(Comparator.comparingDouble(EstadoRuta::ocupacion).reversed()
                        .thenComparing(EstadoRuta::ruta))
                .toList();
```

```
    }

    // ------------------------------------------------------------------
    // Funcion generica reutilizable
    // ------------------------------------------------------------------

    /**
     * Las n parejas (llave, cantidad) con mayor cantidad. Si hay empate, ordena por la llave.
     * Es generica: sirve para estaciones, horas, rutas, etc.
     */
    public static <K extends Comparable<? super K>> List<Map.Entry<K, Long>> topN(Map<K, Long>
conteos, int n) {
        return conteos.entrySet().stream()
                .sorted(Map.Entry.<K, Long>comparingByValue().reversed()
                        .thenComparing(Map.Entry.<K, Long>comparingByKey()))
                .limit(n)
                .map(e -> Map.entry(e.getKey(), e.getValue()))
                .toList();
    }
}
```

src/tecnomovil/reportes/Reporte.java

```
package tecnomovil.reportes;

import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.ResultadoTiempos;
import tecnomovil.modelo.Tramo;

import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

/**
 * Convierte los resultados en texto para mostrarlos.
 * Son funciones puras: reciben datos y DEVUELVEN un String; no imprimen nada.
 * Quien imprime es la clase App (el unico lugar con efectos secundarios).
 */
public final class Reporte {

    private Reporte() { }

    private static final int ANCHO_BARRA = 30;

    /** Titulo de una seccion. */
    public static String titulo(String texto) {
        String linea = "=".repeat(72);
        return "\n" + linea + "\n " + texto + "\n" + linea;
    }

    /** Numero con separador de miles (ej: 12.345). */
    public static String numero(long n) {
        return String.format(Locale.GERMANY, "%,d", n);
    }

    /** Barra de texto proporcional al valor maximo. */
    public static String barra(long valor, long maximo) {
        int largo = maximo == 0 ? 0 : (int) Math.round((double) valor / maximo * ANCHO_BARRA);
        return "#".repeat(largo);
    }

    /** Tabla de "llave -> cantidad" con barras. La funcion etiqueta decide como se muestra la llave.
*/
    public static <K> String tablaConBarras(List<Map.Entry<K, Long>> filas, Function<K, String>
```

```
etiqueta) {
        long maximo = filas.stream().mapToLong(Map.Entry::getValue).max().orElse(0);
        return filas.stream()
                .map(e -> String.format("  %-26s %9s  %s",
                        etiqueta.apply(e.getKey()), numero(e.getValue()), barra(e.getValue(),
maximo)))
                .collect(Collectors.joining("\n"));
    }

    /** Muestra "06:00 - 06:59" a partir de la hora 6. */
    public static String rangoHora(int hora) {
        return String.format("%02d:00 - %02d:59", hora, hora);
    }

    /** Patrones de viaje de algunos usuarios. */
    public static String patrones(Map<String, List<String>> patrones, int cuantos) {
        return patrones.entrySet().stream()
                .limit(cuantos)
                .map(e -> "  " + e.getKey() + ": " + String.join(" -> ", e.getValue()))
                .collect(Collectors.joining("\n"));
    }

    /** Resumen de tiempos entre estaciones y los tramos mas usados. */
    public static String tiempos(ResultadoTiempos r, int cuantos) {
        String encabezado = String.format(Locale.US,
                "  Desplazamientos analizados: %s%n  Tiempo promedio general: %.1f minutos%n%n"
                        + "  %-45s %8s %10s", numero(r.totalDesplazamientos()),
r.promedioGeneralMinutos(),
                "Tramo (los mas recorridos)", "Viajes", "Promedio");
        String filas = r.viajesPorTramo().entrySet().stream()
                .sorted(Map.Entry.<Tramo, Long>comparingByValue().reversed()
                        .thenComparing(e -> e.getKey().toString()))
                .limit(cuantos)
                .map(e -> String.format(Locale.US, "  %-45s %8s %7.1f min", e.getKey(),
                        numero(e.getValue()), r.promedioPorTramo().get(e.getKey())))
                .collect(Collectors.joining("\n"));
        return encabezado + "\n" + filas;
    }

    /** Tabla de estado de las rutas. */
    public static String estadoRutas(List<EstadoRuta> estados, long capacidad, double umbral) {
        String encabezado = String.format(Locale.US,
                "  Capacidad simulada: %s pasajeros por hora | Umbral: %.0f %%%n%n  %-27s %8s %13s %9s
%s",
                numero(capacidad), umbral * 100, "Ruta", "Usos dia", "Hora pico", "Ocupacion",
"Estado");
        String filas = estados.stream()
                .sorted(Comparator.comparingDouble(EstadoRuta::ocupacion).reversed())
                .map(e -> String.format(Locale.US, "  %-27s %8s %13s %8.1f%%  %s",
                        e.ruta(), numero(e.usosTotales()), rangoHora(e.horaPico()),
                        e.ocupacion() * 100, e.critica() ? "CRITICA (!)" : "normal"))
                .collect(Collectors.joining("\n"));
        return encabezado + "\n" + filas;
    }
}
```

src/tecnomovil/App.java

```
package tecnomovil;

import tecnomovil.datos.GeneradorDatos;
import tecnomovil.modelo.Accion;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.Registro;
import tecnomovil.modelo.ResultadoTiempos;
import tecnomovil.procesamiento.Filtros;
import tecnomovil.procesamiento.ProcesadorTransporte;
```

```
import tecnomovil.reportes.Reporte;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.function.Supplier;

/**
 * Programa principal de TecnoMovil Data.
 *
 * Esta es la UNICA clase que tiene efectos secundarios (imprimir en pantalla
 * y medir el tiempo). Todo el calculo se hace en ProcesadorTransporte con
 * funciones puras. Asi se separa "calcular" de "mostrar".
 *
 * Uso: java -cp out tecnomovil.App [cantidadDeUsuarios]
 */
public final class App {

    private static final long SEMILLA = 2026L;
    private static final int USUARIOS_POR_DEFECTO = 150_000;
    private static final long CAPACIDAD_POR_HORA = 4_500L;   // pasajeros por hora por ruta (simulado)
    private static final double UMBRAL_CRITICO = 0.85;        // 85 % de la capacidad

    /** Resultado de una operacion y cuanto se demoro. */
    private record Medicion<T>(T valor, long milisegundos) { }

    private static <T> Medicion<T> medir(Supplier<T> operacion) {
        long inicio = System.nanoTime();
        T valor = operacion.get();
        return new Medicion<>(valor, (System.nanoTime() - inicio) / 1_000_000);
    }

    /** Todos los resultados juntos, para comparar el modo secuencial con el paralelo. */
    private record Resultados(Map<String, Long> afluencia,
                              Map<String, Long> usuariosUnicos,
                              Map<Integer, Long> flujoPorHora,
                              Map<String, Long> usosPorRuta,
                              Map<String, List<String>> patrones,
                              ResultadoTiempos tiempos,
                              List<EstadoRuta> estadoRutas) { }

    /** Ejecuta las seis tareas (a-f) sobre la misma lista. Es una funcion pura. */
    private static Resultados procesarTodo(List<Registro> registros, boolean paralelo) {
        return new Resultados(
                ProcesadorTransporte.afluenciaPorEstacion(registros, paralelo),
                ProcesadorTransporte.usuariosUnicosPorEstacion(registros, paralelo),
                ProcesadorTransporte.flujoPorHora(registros, paralelo),
                ProcesadorTransporte.usosPorRuta(registros, paralelo),
                ProcesadorTransporte.patronesPorUsuario(registros, paralelo),
                ProcesadorTransporte.tiemposEntreEstaciones(registros, paralelo),
                ProcesadorTransporte.estadoDeRutas(registros, CAPACIDAD_POR_HORA, UMBRAL_CRITICO,
paralelo));
    }

    public static void main(String[] args) {
        int usuarios = args.length > 0 ? Integer.parseInt(args[0]) : USUARIOS_POR_DEFECTO;

        System.out.println(Reporte.titulo("TECNOMOVIL DATA - Procesamiento funcional del transporte de
TecnoValle"));
        System.out.println("  Java " + System.getProperty("java.version")
                + " | Nucleos del procesador: " + Runtime.getRuntime().availableProcessors());

        // 1. Generar los datos simulados -------------------------------------------------
        Medicion<List<Registro>> generacion = medir(() -> GeneradorDatos.generar(usuarios, SEMILLA));
        List<Registro> registros = generacion.valor();
        int huellaAntes = registros.hashCode();
        System.out.println("  Fecha simulada: " + GeneradorDatos.FECHA
                + " | Usuarios: " + Reporte.numero(usuarios)
                + " | Registros generados: " + Reporte.numero(registros.size())
                + " (" + generacion.milisegundos() + " ms)");
```

```
        // 2. Comprobar que la lista original no se puede modificar ----------------------
        System.out.println(Reporte.titulo("INMUTABILIDAD - Intento de modificar la lista original"));
        try {
            registros.add(new Registro("U999999", "Ruta falsa", "Estacion falsa", Accion.ENTRADA,
                    LocalDateTime.now()));
            System.out.println("  ERROR: la lista se dejo modificar");
        } catch (UnsupportedOperationException e) {
            System.out.println("  OK: Java no deja agregar registros ->
UnsupportedOperationException");
        }

        // 3. Procesar en modo secuencial -----------------------------------------------
        Medicion<Resultados> secuencial = medir(() -> procesarTodo(registros, false));
        Resultados r = secuencial.valor();

        System.out.println(Reporte.titulo("a) AFLUENCIA POR ESTACION (top 10 por entradas)"));
        System.out.println(String.format("  %-26s %9s", "Estacion", "Entradas"));
        System.out.println(Reporte.tablaConBarras(ProcesadorTransporte.topN(r.afluencia(), 10), e ->
e));
        System.out.println("\n  Estacion con mas usuarios distintos: "
                + ProcesadorTransporte.topN(r.usuariosUnicos(), 1).get(0).getKey()
                + " (" + Reporte.numero(ProcesadorTransporte.topN(r.usuariosUnicos(),
1).get(0).getValue())
                + " usuarios)");

        System.out.println(Reporte.titulo("b) HORAS PICO (flujo de entradas + salidas por hora)"));
        System.out.println(Reporte.tablaConBarras(List.copyOf(r.flujoPorHora().entrySet()),
Reporte::rangoHora));
        System.out.println("\n  Las 3 horas pico: " + ProcesadorTransporte.horasPico(r.flujoPorHora(),
3).stream()
                .map(e -> Reporte.rangoHora(e.getKey()) + " (" + Reporte.numero(e.getValue()) + ")")
                .reduce((x, y) -> x + ", " + y).orElse(""));
        long entradasPicoManana = registros.stream().filter(Filtros.ENTRADA_PICO_MANANA).count();
        System.out.println("  Entradas entre 6:00 y 8:59 (predicados compuestos): "
                + Reporte.numero(entradasPicoManana));

        System.out.println(Reporte.titulo("c) RUTAS MAS UTILIZADAS"));
        System.out.println(Reporte.tablaConBarras(ProcesadorTransporte.topN(r.usosPorRuta(), 8), e ->
e));

        System.out.println(Reporte.titulo("d) PATRONES DE VIAJE POR USUARIO (primeros 8 de "
                + Reporte.numero(r.patrones().size()) + ")"));
        System.out.println(Reporte.patrones(r.patrones(), 8));

        System.out.println(Reporte.titulo("e) TIEMPO PROMEDIO ENTRE ESTACIONES CONSECUTIVAS"));
        System.out.println(Reporte.tiempos(r.tiempos(), 8));

        System.out.println(Reporte.titulo("f) DETECCION DE SOBRECARGA EN RUTAS"));
        System.out.println(Reporte.estadoRutas(r.estadoRutas(), CAPACIDAD_POR_HORA, UMBRAL_CRITICO));
        System.out.println("\n  Rutas criticas: " + r.estadoRutas().stream()
                .filter(EstadoRuta::critica).map(EstadoRuta::ruta).toList());

        // 4. Procesar en paralelo y comparar -------------------------------------------
        Medicion<Resultados> paralelo = medir(() -> procesarTodo(registros, true));
        boolean mismosResultados = r.equals(paralelo.valor());

        System.out.println(Reporte.titulo("PARALELISMO - stream() contra parallelStream()"));
        System.out.println("  Secuencial: " + Reporte.numero(secuencial.milisegundos()) + " ms |
Paralelo: "
                + Reporte.numero(paralelo.milisegundos()) + " ms");
        System.out.println("  Los dos modos dan exactamente los mismos resultados: " +
(mismosResultados ? "SI" : "NO"));

        // 5. La lista original sigue igual ---------------------------------------------
        boolean listaIntacta = huellaAntes == registros.hashCode();
        System.out.println(Reporte.titulo("VERIFICACION FINAL"));
        System.out.println("  La lista original no cambio despues de todo el proceso: " +
(listaIntacta ? "SI" : "NO"));
        System.out.println("  Registros: " + Reporte.numero(registros.size()));
```

```
    }
}
```

src/tecnomovil/Verificacion.java

```
package tecnomovil;

import tecnomovil.modelo.Accion;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.Registro;
import tecnomovil.modelo.ResultadoTiempos;
import tecnomovil.modelo.Tramo;
import tecnomovil.procesamiento.Filtros;
import tecnomovil.procesamiento.ProcesadorTransporte;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.function.Supplier;

/**
 * Pruebas del modulo SIN frameworks externos.
 *
 * Se usa un conjunto pequeno de registros hechos a mano, donde el resultado
 * correcto se puede calcular con lapiz y papel, y se compara con lo que
 * devuelven las funciones.
 *
 * Uso: java -cp out tecnomovil.Verificacion
 */
public final class Verificacion {

    /** Resultado de una prueba. */
    private record Prueba(String nombre, boolean paso, String detalle) { }

    private static Prueba prueba(String nombre, Object esperado, Supplier<Object> obtenido) {
        Object valor = obtenido.get();
        return new Prueba(nombre, esperado.equals(valor), "esperado=" + esperado + " | obtenido=" +
valor);
    }

    private static LocalDateTime t(int hora, int minuto) {
        return LocalDateTime.of(2026, 10, 30, hora, minuto);
    }

    /** Datos de prueba: 3 usuarios, 2 rutas, 10 registros. */
    private static final List<Registro> DATOS = List.of(
            new Registro("U1", "R1", "A", Accion.ENTRADA, t(7, 0)),
            new Registro("U2", "R1", "A", Accion.ENTRADA, t(7, 10)),
            new Registro("U1", "R1", "C", Accion.SALIDA, t(7, 20)),
            new Registro("U3", "R2", "B", Accion.ENTRADA, t(7, 30)),
            new Registro("U2", "R1", "B", Accion.SALIDA, t(7, 40)),
            new Registro("U3", "R2", "D", Accion.SALIDA, t(8, 0)),
            new Registro("U1", "R2", "C", Accion.ENTRADA, t(17, 0)),
            new Registro("U1", "R2", "D", Accion.SALIDA, t(17, 30)),
            new Registro("U2", "R1", "B", Accion.ENTRADA, t(18, 0)),
            new Registro("U2", "R1", "A", Accion.SALIDA, t(18, 10))
    );

    public static void main(String[] args) {
        int huellaAntes = DATOS.hashCode();
        ResultadoTiempos tiempos = ProcesadorTransporte.tiemposEntreEstaciones(DATOS, false);

        List<Prueba> pruebas = List.of(
            prueba("a) Afluencia por estacion",
                Map.of("A", 2L, "B", 2L, "C", 1L),
                () -> ProcesadorTransporte.afluenciaPorEstacion(DATOS, false)),
            prueba("a) Usuarios unicos por estacion",
```

```
                Map.of("A", 2L, "B", 2L, "C", 1L),
                () -> ProcesadorTransporte.usuariosUnicosPorEstacion(DATOS, false)),
            prueba("b) Flujo por hora",
                Map.of(7, 5L, 8, 1L, 17, 2L, 18, 2L),
                () -> ProcesadorTransporte.flujoPorHora(DATOS, false)),
            prueba("b) Hora pico",
                7,
                () -> ProcesadorTransporte.horasPico(ProcesadorTransporte.flujoPorHora(DATOS, false),
1).get(0).getKey()),
            prueba("c) Usos por ruta",
                Map.of("R1", 3L, "R2", 2L),
                () -> ProcesadorTransporte.usosPorRuta(DATOS, false)),
            prueba("d) Patron del usuario U1",
                List.of("A", "C", "C", "D"),
                () -> ProcesadorTransporte.patronesPorUsuario(DATOS, false).get("U1")),
            prueba("d) Patron del usuario U2",
                List.of("A", "B", "B", "A"),
                () -> ProcesadorTransporte.patronesPorUsuario(DATOS, false).get("U2")),
            // Desplazamientos: U1 A->C 20 min, U1 C->D 30 min, U2 A->B 30 min, U2 B->A 10 min, U3 B-
>D 30 min
            prueba("e) Cantidad de desplazamientos",
                5L,
                tiempos::totalDesplazamientos),
            prueba("e) Tiempo promedio general (minutos)",
                24.0,
                tiempos::promedioGeneralMinutos),
            prueba("e) Tiempo promedio del tramo A -> C",
                20.0,
                () -> tiempos.promedioPorTramo().get(new Tramo("A", "C"))),
            // Entradas por ruta y hora: R1 -> 7h: 2, 18h: 1 | R2 -> 7h: 1, 17h: 1. Capacidad 2,
umbral 0.75
            prueba("f) Rutas criticas (capacidad 2, umbral 75 %)",
                List.of("R1"),
                () -> ProcesadorTransporte.estadoDeRutas(DATOS, 2, 0.75, false).stream()
                        .filter(EstadoRuta::critica).map(EstadoRuta::ruta).toList()),
            prueba("f) Ocupacion de R1 en su hora pico",
                1.0,
                () -> ProcesadorTransporte.estadoDeRutas(DATOS, 2, 0.75, false).get(0).ocupacion()),
            prueba("Predicados compuestos: entradas entre 6:00 y 8:59",
                3L,
                () -> DATOS.stream().filter(Filtros.ENTRADA_PICO_MANANA).count()),
            prueba("Paralelo da lo mismo que secuencial (patrones)",
                ProcesadorTransporte.patronesPorUsuario(DATOS, false),
                () -> ProcesadorTransporte.patronesPorUsuario(DATOS, true)),
            prueba("Inmutabilidad: el resultado no se puede modificar",
                "UnsupportedOperationException",
                () -> {
                    try {
                        ProcesadorTransporte.afluenciaPorEstacion(DATOS, false).put("Z", 1L);
                        return "se modifico";
                    } catch (UnsupportedOperationException e) {
                        return "UnsupportedOperationException";
                    }
                }),
            prueba("Inmutabilidad: la lista original no cambio",
                huellaAntes,
                DATOS::hashCode)
        );

        System.out.println("PRUEBAS DEL MODULO TECNOMOVIL DATA (sin frameworks)\n");
        pruebas.forEach(p -> System.out.println((p.paso() ? "  [OK]    " : "  [FALLA] ") + p.nombre()
                + (p.paso() ? "" : "\n          " + p.detalle())));

        long aprobadas = pruebas.stream().filter(Prueba::paso).count();
        System.out.println("\n  Resultado: " + aprobadas + " de " + pruebas.size() + " pruebas
aprobadas");
        if (aprobadas != pruebas.size()) {
            System.exit(1);
        }
```

```
    }
}
```